CSA17 – Analytical Problem-Solving Report

Artificial Intelligence – Assessment Tool 1

Name: Hemalakshmi H

Reg No: 192571049

Q1. Water Jug Problem (4-gallon & 3-gallon)

Problem Explanation

You are given two jugs:

- Jug A = 4 gallons
- Jug B = 3 gallons

Goal: **Get exactly 2 gallons in the 4-gallon jug** using operations: Fill, Empty, Pour.

Algorithm

1. Fill 3-gallon jug.
2. Pour into 4-gallon jug.
3. Fill 3-gallon jug again.
4. Pour until 4-gallon jug becomes full.
5. Empty 4-gallon jug.
6. Pour remaining 2 gallons into 4-gallon jug.

Python Code

```
def water_jug():
    four = 0
    three = 0
    steps = []

    three = 3
    steps.append((four, three))
```

```
    four += three
    three = 0
    steps.append((four, three))

    three = 3
    steps.append((four, three))

    space = 4 - four
    three -= space
    four = 4
    steps.append((four, three))

    four = 0
    steps.append((four, three))

    four = three
    three = 0
    steps.append((four, three))

    return steps
```

Output

Final state: **4-gallon jug = 2 gallons**

Q2. Mars Rover Intelligent Agent

Concept Explanation

Mars Rover is an **autonomous intelligent agent** operating in a harsh, partially observable environment.

Percepts

- Camera images
- Soil composition
- Chemical readings
- Temperature
- Terrain
- Obstacles
- Wheel slip
- Position sensors

Actions

- Move
- Turn
- Capture images
- Drill
- Analyze samples
- Transmit data
- Adjust solar panels

Performance Measures

- Samples collected
- Energy efficiency
- Hazard avoidance
- Navigation accuracy
- Mission goals achieved

Suitable Agent Architecture

Hybrid agent (Reactive + Deliberative)

- Reactive → obstacle avoidance
- Deliberative → long-term planning

Python Code

class MarsRover:

```
def __init__(self):
    self.energy = 100
    self.samples_collected = 0

def percepts(self):
    return {"camera": "image", "soil": "data", "temp": -60}

def actions(self):
    return ["move", "turn", "capture", "drill"]

def move(self):
    self.energy -= 5

def collect_sample(self):
    self.samples_collected += 1
    self.energy -= 10
```

Q3. 8-Queens Problem

Problem Explanation

Place 8 queens on a chessboard such that **no two queens attack each other**.

Search Components

- **State:** Positions of queens
- **Initial State:** Empty board
- **Successor Function:** Place queen in next column
- **Goal Test:** 8 queens placed safely
- **Strategy:** Backtracking

Python Code

```
N = 8


def is_safe(board, row, col):
    for i in range(col):
        if board[i] == row or abs(board[i] - row) == abs(i - col):
            return False
    return True


def solve_queens():
    board = [-1] * N
    solutions = []


    def backtrack(col):
        if col == N:
            solutions.append(board[:])
            return
        for row in range(N):
            if is_safe(board, row, col):
                board[col] = row
                backtrack(col + 1)


    backtrack(0)
    return solutions
```

Output

Total solutions: **92** First solution: e.g., [0, 4, 7, 5, 2, 6, 1, 3]

Q4. OLA Cab Booking Agent

Problem Explanation

Customer selects cab type → agent chooses best cab based on ETA.

Agent Type

Goal-based agent Goal: Book best cab based on availability + ETA.

Python Code

```
class Cab:
    def __init__(self, cab_type, eta):
        self.cab_type = cab_type
        self.eta = eta


def get_available_cabs():
    return [
        Cab("mini", 5),
        Cab("sedan", 3),
        Cab("micro", 7),
        Cab("prime", 2)
    ]


def book_cab(source, destination, preferred_type):
    cabs = get_available_cabs()
    filtered = [c for c in cabs if c.cab_type == preferred_type]
    if not filtered:
        filtered = cabs
    best = min(filtered, key=lambda x: x.eta)
    fare = best.eta * 10
```

```
    return f"Cab booked: {best.cab_type}, ETA={best.eta}, Fare={fare}"
```

Output

Cab booked: **sedan**, ETA: **3 mins**, Fare: **30 INR**

Q5. Uniform Cost Search (UCS) – Logistics Network

Problem Explanation

Find least-cost path from **S → G** in weighted graph.

Python Code

```
import heapq

graph = {
    'S': [('A', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': []
}

def ucs(start, goal):
    pq = [(0, start, [])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)
```

```
        if node in visited:
            continue
        visited.add(node)
        path = path + [node]
        if node == goal:
            return cost, path
        for neighbor, weight in graph[node]:
            heapq.heappush(pq, (cost + weight, neighbor, path))

cost, path = ucs('S', 'G')
```

Output

Least-cost path: **S → A → C → G** Total cost: **4**